Unit 1 : Introduction to Python

BT151CO · Object-Oriented Programming

Week 1–3 · 6 Hours · CLO 1

Start here. Everything begins with Python.

Prepared by Er. Bishnu Khadka

📋 What We'll Cover

Unit 1 — 6 Hours across 3 Weeks

- What is Python and why it matters
- Variables and Data Types
- String Operations
- Input and Output
- Output Formatting with f-strings
- Operators — Arithmetic, Comparison, Logical
- Program Flow and Indentation
- Decision Making: if / elif / else
- Loops: while and for
- range(), break, continue

🎯 Learning Objectives

By the end of Unit 1, you will be able to:

- Write syntactically correct Python programs
- Use variables, data types, and operators confidently
- Read and write string data using built-in methods
- Accept user input and format output cleanly
- Control program flow using if, while, and for
- Use range(), break, and continue effectively

Apply basic Python syntax to solve structured problems

Section 1

What is Python?

And why should you care?

What is Python?

- High-level, interpreted programming language
- Created by Guido van Rossum — released 1991
- Powers: Instagram, YouTube, NASA, Netflix, WhatsApp
- Ranked #1 most popular language (TIOBE 2024–2025)
  - High-level → write near-English, not machine code
  - Interpreted → run immediately, no compilation needed
  - Readable → code reads like structured English

Feature	Benefit
High-level	Near-English syntax
Interpreted	Instant execution
Open source	Free forever
Huge ecosystem	1M+ libraries

🤔 Classroom Question: Name one app or website you use daily that is built with Python.

Your First Python Program

Live — type this yourself!

```
# This is a comment — Python ignores it
print("Hello, World!")

# Output:
# Hello, World!
```

- # starts a comment — Python ignores the rest of that line
- print() is a built-in function — displays output on screen
- "Hello, World!" is a string — text wrapped in quotation marks
- Every Python program starts with at least one print()

🤔 Think: What happens if you type print(Hello, World!) — without the quotes?

Section 2

Variables & Data Types

Storing information so we can use it later

📦 Variables — Labelled Storage Boxes

Name a value so you can use it later

```
# Store a student's information
student_name = "Alice"   # text
student_age  = 19        # whole number
gpa          = 3.75      # decimal
is_enrolled  = True      # yes/no

print(student_name, "| Age:", student_age)
print("GPA:", gpa, "| Enrolled:", is_enrolled)
```

- variable_name = value → the assignment syntax
- Names use snake_case: all lowercase, underscores between words
- Python is CASE-SENSITIVE: score ≠ Score ≠ SCORE
- No need to declare the type — Python figures it out

💡 Analogy: A variable is a labelled box. The name is on the outside; the value is inside.

🤔 Think: If student_name = 'Alice' and Student_name = 'Bob', what does print(student_name) show?

🏷️ Python's Four Basic Data Types

Type	Keyword	Example	Use For
Integer	int	85, -3, 0	Scores, counts, ages
Decimal	float	9.5, 3.14	Prices, averages, GPA
Text	str	"Alice"	Names, messages, labels
Yes / No	bool	True, False	Pass/fail, on/off

```
score    = 85          # int
average  = 87.5        # float
name     = "Alice"     # str
enrolled = True        # bool

print(type(score))
print(type(name))
# <class 'int'>
# <class 'str'>
```

🤔 Think: What type is the value '85' (with quotes)? What about 85 (no quotes)?

🔄 Type Conversion

Converting between types explicitly

```
# input() ALWAYS returns a string
score_text = input("Enter score: ")
# User types: 85

# MUST convert before doing maths
score = int(score_text)
bonus = 5
final = score + bonus
print(f"Final: {final}")   # 90
```

- int(x) → Convert to integer: int('85') → 85
- float(x) → Convert to decimal: float('9.5') → 9.5
- str(x) → Convert to text: str(100) → '100'
- bool(x) → Convert to True/False: bool(0) → False

❗ Common Mistake: "85" + 5 → TypeError! Always use int("85") + 5 instead.

🤔 Think: What does float('3') give? What about int('3.5') — will it work?

Section 3

String Operations

Working with text in Python

🔤 Strings — Indexing & Slicing

Every character has a position number starting at 0

	A	l	i	c	e
Index:	0	1	2	3	4
Neg:	-5	-4	-3	-2	-1

```
name = "Alice"
print(name[0])     # A   — first character
print(name[-1])    # e   — last character
print(name[1:4])   # lic — index 1, 2, 3 (NOT 4)
print(name[:3])    # Ali — from start to index 2
print(name[2:])    # ice — from index 2 to end
```

🤔 Think: What does name[0:5] give? What about name[5]? (Hint: 'Alice' has indices 0–4)

🛠️ Essential String Methods

Built-in tools for manipulating text

```
course = "  Introduction to Python  "

print(course.strip())          # 'Introduction to Python'
print(course.strip().upper())  # 'INTRODUCTION TO PYTHON'
print(course.strip().lower())  # 'introduction to python'
print(course.strip().title())  # 'Introduction To Python'

name = "Hello, Alice!"
print(name.replace("Alice", "Bob"))  # 'Hello, Bob!'

csv = "Alice,85,19"
print(csv.split(","))              # ['Alice', '85', '19']
print(len("Python"))               # 6
```

⚠️ Strings are IMMUTABLE — methods return a NEW string; the original is unchanged!

Section 4

Input, Output & Formatting

Communicating with the user

Input and Output

Communicating with the user

```
# Output with options
print("Alice", 85, sep=" | ")     # Alice | 85
print("Loading", end="...")       # no newline at end
print("Done!")                     # Loading...Done!

# Input — ALWAYS returns a STRING
name  = input("Enter your name: ")
score = int(input("Enter score: "))  # convert!

print(f"Welcome, {name}. Score: {score}")
```

- sep= changes the separator between printed values (default: space)
- end= changes what prints at the end of the line (default: newline)
- input() ALWAYS returns a string — convert to int/float as needed

! Forgetting to convert input() is the #1 TypeError source for beginners!

🎨 Output Formatting — f-Strings

The modern, preferred way to format output in Python 3.6+

```
student_name = "Alice"
score        = 87.333

print(f"Student : {student_name}")
print(f"Score   : {score:.2f}/100")     # 2 decimal places → 87.33
print(f"Grade   : {'A' if score >= 85 else 'B'}")

# Alignment and number formatting
print(f"{'Title':<25} {'Year':>6}")
print(f"{'Python Crash Course':<25} {2019:>6}")
print(f"Balance: ₹{1234567:,.2f}")
```

- Prefix the string with f — then put variables or expressions inside {}
- :.2f → float to 2 decimal places
- :, → number with comma separator → 1,234,567
- :<N → left-align in N characters :>N → right-align

Section 5

Operators

Making calculations and decisions

➕ Arithmetic Operators

Making calculations

Op	Meaning	Example	Result
+	Addition	`10 + 3`	13
-	Subtraction	`10 - 3`	7
*	Multiplication	`10 * 3`	30
/	Division	`10 / 3`	3.333
//	Floor Division	`10 // 3`	3
%	Remainder	`10 % 3`	1
**	Power	`2 ** 8`	256

```
price    = 120.0
quantity = 3
discount = 10   # percent

subtotal = price * quantity
saving   = subtotal * (discount / 100)
final    = subtotal - saving

print(f"Subtotal : ₹{subtotal:.2f}")
print(f"Saving   : ₹{saving:.2f}")
print(f"Total    : ₹{final:.2f}")
# Subtotal : ₹360.00
# Total    : ₹324.00
```

🤔 Think: If 17 books are shared equally among 5 students, then how many each? How many left over? → 17//5=3, 17%5=2

⚖️ Comparison & Logical Operators

Making decisions — results are always True or False

```
score      = 85
attendance = 90
submitted  = True

# AND: ALL must be True
eligible = (score >= 75
            and attendance >= 80
            and submitted)
print(f"Eligible: {eligible}")   # True

# OR: at least ONE True
top = score >= 95 or attendance == 100
print(f"Top student: {top}")     # False
```

Op	Meaning
==	Equal
!=	Not equal
>	Greater
<	Less
>=	≥
<=	≤

Logical:

`and` – both True

`or` – one True

`not` – reverses

❗ Common Mistake: = assigns a value. == compares two values. NEVER use = inside an if condition!

🤔 Score=80, Attendance=75. Is the student eligible if both must be ≥75 and ≥80?

Section 6

Program Flow & Indentation

Structure determines behaviour

📐 Indentation — Python's Most Important Rule

Structure is determined by spaces, not curly braces

```
# ❌ WRONG — IndentationError
if score >= 75:
print("Pass")    # No indent!
  print("Done")  # Wrong indent
```

```
# ✅ CORRECT — 4 spaces per level
if score >= 75:
    print("Pass")     # 4 spaces
    print("Done")     # 4 spaces
print("Always runs")  # 0 spaces
```

- Use exactly 4 spaces per indentation level
- NEVER use tabs — configure VS Code to insert spaces
- All lines in the same block must have the same indent
- A single extra or missing space will crash the program
- VS Code shows indent guides — keep them aligned

🤔 In the correct code above — which lines run if score is 60? → Only 'Always runs'

Section 7

Decision Making

if / elif / else

🔀 The if Statement

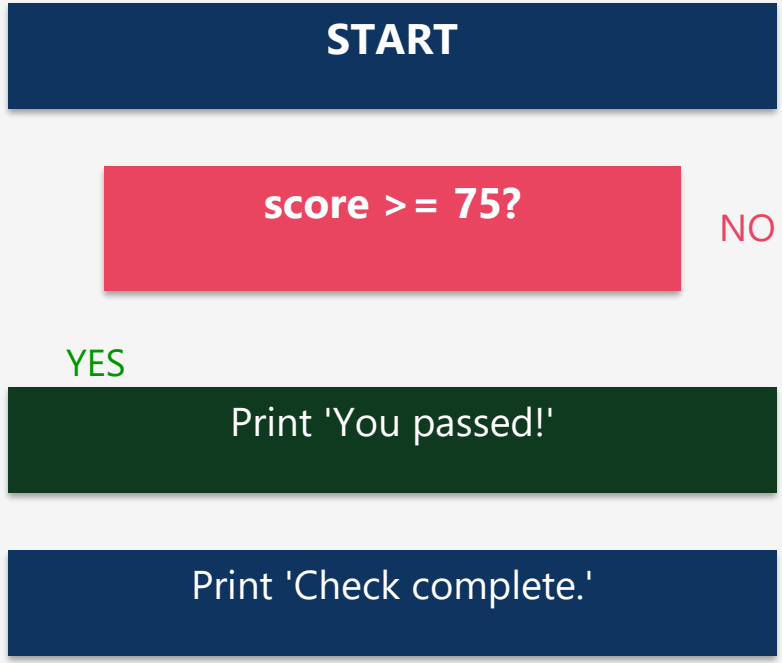
Execute a block only when a condition is True

```
score = 85

if score >= 75:
    print("✅ You passed!")
    print("   Keep it up.")

print("Check complete.")  # always runs

# Output:
# ✅ You passed!
#    Keep it up.
# Check complete.
```



- Condition must evaluate to True or False
- Don't forget the colon : at the end of if line
- Indented block runs ONLY when condition is True
- Code after the block ALWAYS runs

🤔 Think: What does the program print if score = 60? → Only 'Check complete.'

🔀 if / elif / else — Grade Classifier

Python checks conditions top to bottom — the FIRST True match wins

```
score = 78

if score >= 90:
    grade, remark = "A", "Outstanding"
elif score >= 80:
    grade, remark = "B", "Excellent"
elif score >= 70:
    grade, remark = "C", "Good"
elif score >= 60:
    grade, remark = "D", "Satisfactory"
else:
    grade, remark = "F", "Needs Improvement"

print(f"Grade: {grade} — {remark}")
# Grade: C — Good
```

Condition	Result	Action
78 >= 90?	False	Skip
78 >= 80?	False	Skip
78 >= 70?	True ✅	Execute → C
(rest)	—	Skipped

⚠️ Common Mistake: elif after else causes SyntaxError — else must ALWAYS be last!

🤔 If score = 80 — does the student get B or C? → B, because 80 >= 80 is True and Python stops there.

Section 8

Loops

while and for

🔄 The while Loop

Repeat a block as long as a condition remains True

```
# Countdown
count = 5

while count > 0:
    print(f"Launching in {count}...")
    count -= 1    # ← IMPORTANT!

print("🚀 Launch!")

# Launching in 5...
# Launching in 4...
# ...
# 🚀 Launch!
```

count	cond?	Output	after
5	True	Launching in 5...	4
4	True	Launching in 4...	3
3	True	Launching in 3...	2
2	True	Launching in 2...	1
1	True	Launching in 1...	0
0	False ❌	(loop ends)	—

- while condition: → block runs while condition is True
- ALWAYS update the condition variable inside the loop!

❗ Forgetting count -= 1 creates an INFINITE LOOP! Press Ctrl+C to stop.

🔁 The for Loop

Iterate over each item in a known collection

```
# Iterate over a list
students = ["Alice", "Bob", "Charlie", "Diana"]

for student in students:
    print(f"Good morning, {student}!")

# Accumulator pattern — calculate class average
scores = [85, 72, 91, 68, 88]
total  = 0

for score in scores:
    total += score              # add each score to total

print(f"Class average: {total / len(scores):.1f}")   # 80.8
```

- for item in collection: → item takes each value in turn
- Loop runs exactly once per item in the collection
- Accumulator: start total = 0, add each value, divide at the end
- Use for when you know what you're iterating over
- Use while when you don't know how many repetitions you need

🔢 The range() Function

Generate a sequence of numbers without storing them in memory

Form	Example	Generates
range(stop)	`range(5)`	`0, 1, 2, 3, 4`
range(start, stop)	`range(1, 6)`	`1, 2, 3, 4, 5`
range(start, stop, step)	`range(0, 10, 2)`	`0, 2, 4, 6, 8`
range(start, stop, -step)	`range(10, 0, -1)`	`10, 9, …, 1`

```
# Question numbers 1–5
for q in range(1, 6):
    print(f"Question {q}")

# Even numbers 2–10
for n in range(2, 11, 2):
    print(n, end=" ")
# 2 4 6 8 10

# Countdown
for n in range(5, 0, -1):
    print(n, end=" ")
print("Go!")
# 5 4 3 2 1 Go!
```

❗ range(1, 5) → 1,2,3,4 — stop value is NEVER included!

🤔 How many times does range(3, 10, 2) iterate? What does it generate?

✋ break and continue

Fine-grained control over loop execution

```
# break — exit the loop immediately
students = ["Alice","Bob","Charlie"]
target = "Charlie"

for s in students:
    if s == target:
        print(f"✅ Found: {s}")
        break          # stop searching
    print(f"Checking {s}...")

# Checking Alice...
# Checking Bob...
# ✅ Found: Charlie
```

```
# continue — skip this iteration
scores = [85, -1, 72, -1, 91]
names  = ["Alice","Bob","Charlie","Diana","Eve"]

for i in range(len(scores)):
    if scores[i] == -1:
        continue   # skip absent
    print(f"{names[i]}: {scores[i]}")

# Alice: 85
# Charlie: 72
# Eve: 91
```

- break — exit the entire loop immediately (like finding a key)
- continue — skip the rest of THIS iteration, go to the next one
- Both work identically in while loops

💡 break = stop the music entirely. continue = skip this track, play the next one.

🐛 Spot the Bug

Find and fix the error in each snippet — discuss with a partner (60 seconds)

```
# Bug 1
score = input("Enter score: ")
result = score + 10
print(result)
```

Bug 1: TypeError

`Fix: int(input(...))`

```
# Bug 2
if score = 85:
    print("Match!")
```

Bug 2: SyntaxError

`Fix: if score == 85:`

```
# Bug 3
for i in range(5):
print(i)
```

Bug 3: IndentationError

Fix: indent print(i) 4 spaces

💡 Error messages are your friends — they tell you exactly what went wrong and where!

✅ Run each bug in VS Code — train yourself to read error messages before Googling.

🔗 Putting It All Together

A student grade report — using every concept from Unit 1

```
student_name = input("Enter student name: ")
score        = float(input("Enter score (0-100): "))

if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
elif score >= 60:
    grade = "C"
else:
    grade = "F"

passed = score >= 60

print()
print("=" * 35)
print(f"  Student : {student_name}")
print(f"  Score   : {score:.1f}/100")
print(f"  Grade   : {grade}")
print(f"  Result  : {'PASS ✅' if passed else 'FAIL
❌'}")
print("=" * 35)
```

🤔 Challenge:
How would you modify this so it loops and keeps accepting students until the user types 'quit'?

📋 Unit 1 — Key Takeaways

10 things to remember

- Python runs top to bottom, one line at a time — sequential execution
- Every value has a type: int, float, str, bool — know which to use
- input() ALWAYS returns a string — convert before calculating
- Indentation IS syntax — 4 spaces per level, always
- f-strings are the modern, preferred way to format output
- Use for for known sequences; while for unknown repetitions
- break exits a loop; continue skips one iteration
- range(1, 5) generates 1, 2, 3, 4 — stop value is NEVER included
- Strings are immutable — methods return a NEW string
- PEP 8 from day one: snake_case, 4 spaces, clear meaningful names

🎟 Exit Ticket

Write your answers — 3 minutes, then we review together

```
# Q1 — What does this print?
total = 0
for i in range(1, 6):
    if i % 2 == 0:
        continue
    total += i
print(total)
```

Q2 — Short Answer:

Why does input() always return a string?

What would you write to ask a user for their age as an integer?

✅ Q1 Answer: 9 (adds 1+3+5, skips 2 and 4 via continue)

✅ Q2: Python can't know the type in advance. Use age = int(input('Age: '))

Thank You!

Unit 1 — Introduction to Python · BT151CO

```
python --version  →  Python 3.12.x  ✅
```

Prepared by Er. Bishnu Khadka